

KARNATAKA STATE POLICE
DATATHON 2026

Intelligent Conversational AI for the KSP Crime Database

Product Requirements Document

Challenge 1 — Intelligent Conversational AI for KSP Crime Database
State Crime Records Bureau (SCRB) · 1,100+ Police Stations · Karnataka
Version 1.0 | July 2026

*Prepared for the Datathon 2026 submission review.
Contains illustrative schema references only — no real case, victim, or accused data is included.*

Table of Contents

1. Executive Summary	3
2. Vision Statement	4
3. Problem Analysis	5
4. User Personas	6
5. Use Cases	7
6. Product Scope	8
7. Functional Requirements	9
8. Conversational AI Design	12
9. Database Understanding	14
10. Analytics Engine	16
11. System Architecture	17
12. API Design	18
13. Security & Compliance	19
14. Explainable AI Strategy	20
15. UI/UX Specification	21
16. Deployment Strategy	22
17. Hackathon MVP Plan (48 Hours)	23
18. Demo Script (5 Minutes)	24
19. Judging Alignment Matrix	25
20. Final Recommendation	26

1. Executive Summary

The State Crime Records Bureau (SCRB) holds the FIR-level data of over 1,100 police stations across Karnataka inside a well-normalized relational schema (CaseMaster, Accused, Victim, ArrestSurrender, ChargesheetDetails, ActSectionAssociation, CrimeHead/CrimeSubHead, Unit, District, Employee, Court, and their lookup tables). Today, extracting insight from this data means either using a fixed dashboard built for a narrow, pre-defined question, or waiting for a technical team to hand-write SQL against these tables. Neither path supports the kind of ad-hoc, multi-hop investigative question an officer actually has in the field — "has this accused surfaced in another district," "what sections are trending in this crime head this month," "which stations have a disposal backlog."

We propose a conversational AI layer that sits directly on top of this schema: an investigator types or speaks a question in English or Kannada, the system retrieves the relevant tables and columns, generates a constrained, validated SQL query, executes it read-only, and returns an answer together with the exact SQL used, the tables touched, a confidence score, and an audit trail entry. The same platform derives criminal-network graphs from co-accused and repeat-offender patterns in the Accused table, plots geospatial hotspots from CaseMaster.latitude/longitude, and produces simple, explainable trend and anomaly signals for supervisory review.

Impact for KSP: query turnaround drops from days (technical-team SQL requests) to seconds; repeat offenders who currently hide in station-siloed records become visible across districts; and every AI-generated answer carries the provenance required for it to be trusted, reviewed, and challenged — a non-negotiable property for anything touching police data.

Differentiator: this PRD is deliberately schema-grounded — every table, column, and join referenced anywhere in this document is taken directly from the supplied KSP FIR ER diagram, not invented. The NL-to-SQL layer is retrieval-constrained specifically to prevent hallucinated fields, and every feature is scoped against what a small student team can honestly build and demo in 48–72 hours, with a clearly separated roadmap for what a production KSP deployment would additionally require.

2. Vision Statement

With this platform, KSP becomes an intelligence-led force where any Station House Officer, Investigating Officer, or district analyst — in Kannada or English, by typing or by voice — can ask a plain-language question about crime data and receive, within seconds, an answer that is mapped, networked, and cited back to the exact FIR records that produced it. Static dashboards stop being the ceiling of what an officer can ask; the ceiling becomes whatever question the case in front of them demands.

At state level, SCRB and supervisory officers gain a standing, explainable view of cross-district patterns — repeat offenders, emerging hotspots, act/section trends — that today only exists as a slow, manual, one-off analytical exercise.

3. Problem Analysis

3.1 Current workflow limitations

- Dashboards are built for a fixed set of pre-defined slices (e.g. crime-head counts by district). Any question outside that slice requires a new report to be built.
- Ad-hoc, multi-condition questions ("accused appearing in more than 3 FIRs across at least 2 districts, involved in IPC sections related to theft") require someone to hand-write a multi-table join across CaseMaster, Accused, ActSectionAssociation, Unit and District — a task realistically reserved for a technical/reporting team, not the investigating officer who needs the answer.
- Cross-district linkage of the same accused person is not supported today in any systematic way: the schema itself has no state-wide unique person identifier for an accused (Accused.PersonID is only a per-case display label like A1, A2), so identity resolution across FIRs has always been a manual, station-bound exercise.
- Existing dashboard numbers arrive with no attached reasoning — no visible query, no source table, no confidence — so a supervisory officer cannot audit how a number was produced, which limits how much weight it can carry in a review or in court-adjacent discussion.

3.2 Why static dashboards fail investigators

A dashboard answers the question its designer anticipated. Investigation is, by nature, a sequence of questions that each depend on the answer to the last one ("who else was arrested with him" → "has that person surfaced elsewhere" → "under which sections"). Static BI tooling cannot follow that chain without a human rebuilding the query at every hop.

3.3 Why conversational analytics helps

A schema-grounded NL-to-SQL layer generates the exact join needed for the question actually asked, in the officer's own words and language, and can be refined turn-by-turn ("and how many of those were chargesheeted?") without anyone touching SQL. Paired with explainability (the generated SQL, the tables used, and a confidence score returned alongside every answer), it gives investigators dashboard-free flexibility without losing the auditability a policing context requires.

3.4 A weak assumption worth challenging up front

Critical evaluation — socio-demographic and predictive features

The official brief asks for “socio-demographic insights,” “behavioral profiling,” and “predictive analytics.” The schema does expose ComplainantDetails.CasteID and ComplainantDetails.ReligionID. Building predictive or risk-scoring features on protected attributes like caste or religion is both an unnecessary technical choice and a serious fairness/legal exposure — it risks producing discriminatory profiling and is unlikely to survive scrutiny at a public-safety review. This PRD deliberately scopes “socio-demographic insight” down to aggregate, non-individualized victim-support reporting (e.g. district-level counts for resource planning), explicitly excludes caste and religion from every predictive/risk-scoring model (Section 10), and gates even the aggregate view behind SCRB-level RBAC (Section 13). This is a design decision the team should be ready to defend to judges as a strength, not an omission.

4. User Personas

4.1 Station House Officer (SHO)

Goals: Keep the station's FIR register under control — know what is pending, what is aging, what is chargesheet-ready.

Frustrations: No time to write or request SQL reports; needs answers between shift duties.

Typical query: “Show all FIRs at my station pending beyond 60 days, grouped by crime head.”

Primary tables: CaseMaster, CaseStatusMaster, Unit, CrimeHead

4.2 Investigating Officer (IO)

Goals: Build a stronger case file — find co-accused, prior arrests, and cross-district history for a named accused.

Frustrations: Cross-referencing an accused person across other FIRs today means manually asking other stations or SCRB.

Typical query: “Has this accused been arrested before, and in which districts?”

Primary tables: Accused, ArrestSurrender, CaseMaster, Court

4.3 District Crime Analyst

Goals: Track crime-head trends and hotspots for the district; brief the SP with current numbers.

Frustrations: Every new question means exporting to Excel and building a new pivot from scratch.

Typical query: “Show the trend of chain-snatching cases in Mysuru district over the last 6 months.”

Primary tables: CaseMaster, CrimeSubHead, CrimeHead, District, Unit

4.4 SCRB Intelligence Officer

Goals: State-wide criminal-network analysis; surface repeat offenders and act/section patterns across districts.

Frustrations: No systematic cross-district identity resolution exists today; network analysis is done manually, case by case.

Typical query: “Which accused persons appear in more than three FIRs across multiple districts?”

Primary tables: Accused, CaseMaster, District, Unit, ActSectionAssociation

4.5 Superintendent of Police (SP)

Goals: District oversight — resource allocation, early-warning on emerging hotspots, station performance.

Frustrations: Wants a forward-looking signal, not just a historical count, without waiting on a quarterly report.

Typical query: “Which station in my district shows an emerging hotspot for two-wheeler theft?”

Primary tables: CaseMaster, CrimeSubHead, Unit, ChargesheetDetails

5. Use Cases

Twenty-two representative investigative and analytical scenarios, each mapped to the actual schema tables the query engine will join. These double as the acceptance-test query set for the hackathon demo.

ID	Persona	Scenario	Primary Schema Tables
UC-01	SHO	Pending FIR ageing at my station	CaseMaster, CaseStatusMaster, Unit
UC-02	SHO	Officer caseload by crime head	CaseMaster, CrimeHead, Employee
UC-03	IO	Repeat-offender check on a named accused	Accused, CaseMaster
UC-04	IO	Prior arrest/surrender history of an accused	ArrestSurrender, Accused, Court
UC-05	IO	Co-accused network for a specific case	Accused, CaseMaster
UC-06	IO / Analyst	Chargesheet turnaround time by station	ChargesheetDetails, CaseMaster
UC-07	Analyst	Crime hotspot map for a district / date range	CaseMaster (lat/long), CrimeSubHead, Unit
UC-08	Analyst	Trend of a crime sub-head over time	CaseMaster, CrimeSubHead, CrimeHead
UC-09	Analyst	Most-invoked Act/Section combinations	ActSectionAssociation, Act, Section
UC-10	Analyst	Crimes against women by district	CaseMaster, CrimeHead, Victim, District
UC-11	Analyst	Station-wise case disposal comparison	CaseMaster, CaseStatusMaster, Unit
UC-12	SCRB	Cross-district accused name matching	Accused, CaseMaster, Unit, District
UC-13	SCRB	Criminal-network cluster visualization	Accused, CaseMaster (derived graph)
UC-14	SCRB / Analyst	Court-wise pending case load	CaseMaster, Court
UC-15	Analyst	Seasonal / anomaly spike in a crime sub-head	CaseMaster, CrimeSubHead
UC-16	SP	Emerging-hotspot early warning	CaseMaster, CrimeSubHead, Unit
UC-17	SP	False-case / undetected rate by station	ChargesheetDetails (cstype), Unit
UC-18	SP	IO workload balancing across a district	CaseMaster, Employee
UC-19	SCRB	Aggregate victim-demographic pattern for a crime type	Victim, CrimeSubHead (aggregate only)
UC-20	IO	Complainant-also-accused audit flag	ArrestSurrender.IsComplainantAccused, CaseMaster
UC-21	Analyst	Act/Section combinations most linked to heinous cases	ActSectionAssociation, GravityOffence, CaseMaster
UC-22	SCRB	District-to-district accused movement pattern	Accused, Unit, District

6. Product Scope

Prioritization is ruthless on purpose: everything in the MVP list is required for the demo to make its central claim — ask a real investigative question in plain language and get a cited, mapped, networked answer. Everything else is explicitly deferred.

6.1 Must-have MVP (hackathon, 48–72 hrs)

Capability	Tier
English-language NL chat → SQL → answer, with generated SQL + source tables returned on every response	Core
Coverage of CaseMaster, Accused, Victim, ArrestSurrender, ChargesheetDetails, ActSectionAssociation, CrimeHead/CrimeSubHead, Unit, District, Employee, Court, CaseStatusMaster	Core
Repeat-offender detection (fuzzy name + age + gender match across CaseMasterID)	Core
Hotspot map from CaseMaster.latitude/longitude, filterable by crime sub-head and date range	Core
Co-accused network graph for a case, expandable to linked cases	Core
Role-based login (SHO / IO / Analyst / SCRB / SP) with jurisdiction-scoped results	Core
Append-only audit log of every query, its SQL, and the requesting user	Core
PDF export of a single conversation	Core

6.2 Nice-to-have (build only if the MVP is stable)

Capability
Kannada input via a translation pre/post-processing layer (not a native bilingual model)
Voice input via the browser Web Speech API
Simple trend forecasting (moving average / linear regression)
Anomaly detection (z-score on weekly counts)
Multi-turn conversational memory across a session

6.3 Future vision (post-hackathon roadmap, not in scope for the demo)

Capability
Native Kannada-capable LLM, evaluated for on-prem / data-residency-compliant hosting
Migration off consumer cloud (Neon/Vercel/Netlify) to an empanelled government-approved hosting environment for real CCTNS/SCRB data
Graph-database backend (e.g. Neo4j) for state-scale network analysis and centrality queries
Human-in-the-loop, fairness-audited risk scoring with protected attributes formally excluded and logged as excluded

7. Functional Requirements

7.1 Natural-language chat (English)

Description: A chat interface that accepts a free-text question, resolves it against the schema, generates and validates SQL, executes it read-only, and returns a natural-language answer plus a visualization when relevant.

User story: As an Investigating Officer, I want to ask a plain-English question about cases or accused so that I don't need to request a report from a technical team.

Inputs: Free-text question, active user role, active session ID.

Outputs: Natural-language answer, generated SQL, list of tables used, confidence score, optional map/graph/table payload.

Acceptance criteria:

- Answers a question drawn from any of the 22 use cases in Section 5 within 10 seconds on the seeded demo dataset.
- Every response includes the generated SQL and the list of source tables, even when the answer is a single number.
- Out-of-scope or destructive requests (e.g. asking to modify data) are refused with a clear message, not silently ignored.

7.2 Kannada language support

Description: Kannada free-text input is translated to an English-normalized intermediate form before schema-grounded SQL generation; the final natural-language answer is translated back to Kannada.

User story: As a station-level officer more comfortable in Kannada, I want to ask my question in Kannada and read the answer in Kannada.

Inputs: Kannada free-text question.

Outputs: Kannada-language answer; underlying SQL and table list remain in English for auditability.

Acceptance criteria:

- A representative set of 10 Kannada questions (translated equivalents of the Section 5 use cases) return correct answers.
- If Kannada translation confidence is low, the system asks a clarifying question rather than guessing.
- This is explicitly scoped as translation-layer support, not a native Kannada model, and is labelled as such in the UI.

7.3 Voice-enabled interaction

Description: Voice input captured via the browser's Web Speech API, transcribed to text, and passed through the same chat pipeline as typed input.

User story: As an officer with hands occupied or limited typing comfort, I want to speak my question instead of typing it.

Inputs: Microphone audio stream.

Outputs: Transcribed text shown to the user for confirmation before submission; same downstream answer as typed chat.

Acceptance criteria:

- Transcription is shown to the user before the query executes, with an edit option.
- Fails gracefully (falls back to typed input) if the browser does not support the Web Speech API.

7.4 Crime hotspot mapping

Description: A map view that plots CaseMaster incident coordinates as a heatmap/cluster, filterable by district, station, crime sub-head, and date range.

User story: As a District Crime Analyst, I want to see where a crime type is concentrated so I can brief patrol deployment.

Inputs: District/station filter, crime sub-head filter, date range.

Outputs: Heatmap layer, cluster markers, point count per cluster.

Acceptance criteria:

- Map renders correctly for at least three districts in the seeded dataset.
- Clicking a cluster shows the underlying FIR count and a drill-down link to the case list.
- Filters combine (district + crime type + date) without requiring a page reload.

7.5 Criminal network visualization

Description: A graph view built from co-accused relationships (shared CaseMasterID) and repeat-offender links (fuzzy-matched Accused identity across CaseMasterID), rendered with expandable nodes.

User story: As an SCRB Intelligence Officer, I want to see who is connected to whom across cases so I can identify central actors.

Inputs: A starting accused name or CaseMasterID.

Outputs: Graph of nodes (accused persons) and edges (co-accused or repeat-appearance links), each edge labelled with the linking case(s).

Acceptance criteria:

- Expanding a node reveals its direct co-accused within 2 seconds on the seeded dataset.
- Each edge, on hover, shows the CaseMasterID(s) that produced the link — no unexplained edges.
- The identity-matching confidence for repeat-offender links (see Section 9.3) is visibly displayed, not hidden.

7.6 Trend & anomaly / forecasting

Description: Weekly/monthly aggregation of CaseMaster counts by crime sub-head and district, with a simple moving-average trend line and z-score anomaly flagging.

User story: As a Superintendent of Police, I want an early warning when a crime type spikes unexpectedly in my district.

Inputs: District, crime sub-head, lookback window.

Outputs: Trend line chart, list of flagged anomaly weeks with a plain-language reason (e.g. “2.3 standard deviations above the 8-week average”).

Acceptance criteria:

- Anomaly threshold and method are stated in the UI — no black-box “AI predicted” label without a reason.
- Forecast/trend never uses caste, religion, or occupation as an input feature (see Section 10.6).

7.7 PDF export

Description: Export the current chat conversation (questions, answers, generated SQL, and any embedded chart/map image) as a PDF.

User story: As an SHO, I want to save a conversation as a PDF to attach to a briefing note.

Inputs: Active session ID.

Outputs: Downloadable PDF containing the Q&A transcript, SQL used, and a rendered snapshot of any visualization.

Acceptance criteria:

- Exported PDF is generated within 5 seconds for a 10-turn conversation.
- PDF includes the auditable metadata: generating user, role, and timestamp.

7.8 Explainable audit trail

Description: Every query and its generated SQL, requesting user, role, and timestamp is written to an append-only audit log, independently queryable by authorized supervisory roles.

User story: As an SP, I want to review what questions were asked of the crime data and how the AI answered them.

Inputs: None (system-generated on every query).

Outputs: Audit log entries: user, role, timestamp, question text, generated SQL, tables touched, confidence score.

Acceptance criteria:

- No query bypasses audit logging, including failed/rejected queries.
- Audit log is queryable by SCRB/SP roles only (Section 13).

7.9 Role-based secure access

Description: Authentication with a role (SHO / IO / Analyst / SCRB / SP) that scopes both UI features and query results to the user's jurisdiction (Unit/District).

User story: As an SHO, I should not be able to pull case details for a station outside my own without explicit SCRB/SP-level access.

Inputs: Login credentials, assigned role and jurisdiction.

Outputs: Session token; query results automatically filtered to the user's permitted Unit/District scope unless their role is state-wide (SCRB/SP).

Acceptance criteria:

- An SHO's query is silently scoped to their own Unit — no jurisdiction leakage in results.
- SCRB/SP roles can query state-wide, and this state-wide access is itself logged.

8. Conversational AI Design

8.1 Schema-aware retrieval

Every table and column description from the KSP FIR ER diagram (this document's source schema) is embedded into a vector store at build time — table name, column name, type, and the human description supplied in the schema document (e.g. “CrimeMinorHeadID INT FK → CrimeSubHead.CrimeSubHeadID — minor crime sub-head classification”). At query time, the user's question is embedded and the top-k most relevant table/column chunks are retrieved and injected into the SQL-generation prompt. This keeps the model's working context small, current, and grounded in real fields — it is structurally unable to reference a table or column that was not retrieved.

8.2 NL-to-SQL generation

The LLM is prompted with: the retrieved schema subset, the foreign-key relationships between the retrieved tables (taken from the Relationship Matrix in the source ER diagram), 3–5 few-shot examples of question → SQL pairs, and the user's question. It is instructed to produce a single read-only SQL statement and nothing else.

8.3 Query validation

- Parse the generated SQL (e.g. with sqlglot) and reject anything that is not a single SELECT statement.
- Reject any table or column not present in the allow-list built from the retrieved schema chunks.
- Enforce a LIMIT clause (inserted automatically if missing) and a maximum row cap.
- Reject queries that reference ComplainantDetails.CastelID or ComplainantDetails.ReligionID as a filter or GROUP BY key outside the aggregate SCRB reporting endpoint (Section 13.2).

8.4 Safe SQL execution

Validated SQL executes against a database role with SELECT-only privileges on an allow-listed view/table set, with a statement timeout and the row cap from Section 8.3 enforced at the connection level, not just in the prompt.

8.5 Multi-turn memory

The session stores resolved entities from prior turns (e.g. the district or CaseMasterID last referenced) so a follow-up like “and how many of those were chargesheeted?” correctly inherits the filters from the previous turn rather than starting from an empty query.

8.6 Kannada query handling

Kannada input is translated to English before schema retrieval and SQL generation (Section 7.2). This is a deliberate simplification for the hackathon timeline: true Kannada-native schema grounding is listed under Future Vision (Section 6.3).

8.7 Worked examples

Example 1 — “Show robbery hotspots in Bengaluru East in the last 90 days.”

```
SELECT cm.CaseMasterID, cm.latitude, cm.longitude, cm.CrimeRegisteredDate
FROM CaseMaster cm
JOIN CrimeSubHead csh ON cm.CrimeMinorHeadID = csh.CrimeSubHeadID
JOIN Unit u          ON cm.PoliceStationID = u.UnitID
```

```

JOIN District d      ON u.DistrictID = d.DistrictID
WHERE csh.CrimeHeadName = 'Robbery'
      AND d.DistrictName = 'Bengaluru East'
      AND cm.CrimeRegisteredDate >= CURRENT_DATE - INTERVAL '90 days'
LIMIT 2000;

```

Tables used: CaseMaster, CrimeSubHead, Unit, District. Rendered on the map view (Section 15.2).

Example 2 — “Find repeat offenders involved in burglary cases across multiple districts.”

```

SELECT a.AccusedName, a.AgeYear,
       COUNT(DISTINCT d.DistrictID) AS district_count,
       COUNT(DISTINCT cm.CaseMasterID) AS case_count
FROM Accused a
JOIN CaseMaster cm  ON a.CaseMasterID = cm.CaseMasterID
JOIN CrimeSubHead csh ON cm.CrimeMinorHeadID = csh.CrimeSubHeadID
JOIN Unit u        ON cm.PoliceStationID = u.UnitID
JOIN District d    ON u.DistrictID = d.DistrictID
WHERE csh.CrimeHeadName = 'Burglary'
GROUP BY a.AccusedName, a.AgeYear
HAVING COUNT(DISTINCT d.DistrictID) > 1
ORDER BY district_count DESC
LIMIT 500;

```

Tables used: Accused, CaseMaster, CrimeSubHead, Unit, District. Grouping by name+age is a fuzzy proxy for identity — see the limitation flagged in Section 9.3.

Example 3 — “Which Act/Section combinations are most associated with crimes against women in Mysuru?”

```

SELECT s.SectionCode, s.SectionDescription, COUNT(*) AS frequency
FROM ActSectionAssociation asa
JOIN Section s    ON asa.SectionID = s.SectionCode AND asa.ActID = s.ActCode
JOIN CaseMaster cm ON asa.CaseMasterID = cm.CaseMasterID
JOIN Victim v     ON v.CaseMasterID = cm.CaseMasterID
JOIN Unit u       ON cm.PoliceStationID = u.UnitID
JOIN District d   ON u.DistrictID = d.DistrictID
WHERE v.GenderID = :FemaleGenderCode -- resolved from the gender lookup at generation time
      AND d.DistrictName = 'Mysuru'
GROUP BY s.SectionCode, s.SectionDescription
ORDER BY frequency DESC
LIMIT 10;

```

Tables used: ActSectionAssociation, Act, Section, CaseMaster, Victim, Unit, District. The gender filter is resolved via a bound parameter rather than a hard-coded literal, since the exact stored code for GenderID is a lookup value.

9. Database Understanding

9.1 Key entities

- CaseMaster — the hub of the schema; one row per FIR/UDR/PAR/Zero-FIR, holding the crime number, dates, location (latitude/longitude), and foreign keys into every classification and jurisdiction table.
- Victim, Accused, ComplainantDetails — one-to-many children of CaseMaster; a single FIR can have multiple victims, accused, and complainants.
- ArrestSurrender — one-to-many child of CaseMaster, linked back to a specific Accused via AccusedMasterID, and to the Employee who made the arrest (IOID) and the Court the accused was produced before.
- ChargesheetDetails — one row per final report on a case (chargesheet / false case / undetected, via the cstype flag), linked to CaseMaster and the filing officer.
- ActSectionAssociation — many-to-many bridge between CaseMaster and the Act/Section reference tables, recording every legal section invoked on a case.
- CrimeHead / CrimeSubHead — the two-level crime classification (e.g. “Crimes Against Body” → “Murder”) that every FIR is tagged with via CaseMaster.CrimeMajorHeadID / CrimeMinorHeadID.
- Unit → District → State — the jurisdiction hierarchy; every police station is a Unit, self-referencing ParentUnit for command hierarchy, rolling up to District and State.
- Employee — police personnel, carrying Rank, Designation, and current Unit/District posting; referenced as the registering officer on CaseMaster and the arresting officer on ArrestSurrender.
- Court — referenced by both CaseMaster (trial court) and ArrestSurrender (court of production), rolling up to District and State.

9.2 Logical relationships (text form)

CaseMaster is the parent of Victim, Accused, ComplainantDetails, ArrestSurrender, and ActSectionAssociation (all one-to-many). CaseMaster in turn is a child of CaseCategory, GravityOffence, CrimeHead, CrimeSubHead, CaseStatusMaster, Court, and Employee (all many-to-one lookups). ArrestSurrender is additionally a child of State, District, Unit, Court, Employee (as IO), and Accused. ActSectionAssociation bridges CaseMaster to Act and Section, and CrimeHeadActSection separately bridges CrimeHead to Act/Section for classification purposes. ComplainantDetails is a child of OccupationMaster, ReligionMaster, and CasteMaster. The jurisdiction chain Unit → District → State and Court → District → State are both present and used identically for scoping RBAC (Section 13) and for geographic drill-down (Section 15).

9.3 How criminal networks are derived — and a schema limitation to disclose

Important limitation

The schema has no state-wide unique identifier for a person. Accused.PersonID is a per-case display label (A1, A2, A3...), not a person key, and there is no biometric or Aadhaar-linked reference table. Any cross-FIR identity resolution (“is this the same person as in that other case”) is therefore necessarily a fuzzy match on AccusedName + AgeYear + GenderID, and will carry both false positives (common names) and false negatives (spelling variants). Every network edge and repeat-offender flag produced by this system must carry a visible match-confidence score, and the UI must never present a fuzzy-matched identity link as a certainty. This is disclosed here deliberately, and should be stated out loud in the demo (Section 18) — it reads as engineering honesty, not a weakness.

With that caveat stated, the network is built as follows. Nodes: one node per distinct Accused identity cluster (grouped by normalized name + age tolerance + gender, per 9.3 above). Edges: a co-accused edge is drawn between two accused persons

who share the same CaseMasterID; a repeat-appearance edge links two rows in Accused that fuzzy-match to the same identity cluster but sit under different CaseMasterID values. Every edge stores the CaseMasterID(s) that produced it, so the UI can always answer “why is this edge here.”

9.4 Hotspot computation

CaseMaster.latitude/longitude points are filtered by CrimeSubHead and date range, then binned (geohash or fixed-size grid cell) and counted per cell; cell weight can optionally be scaled by GravityOffence to emphasize heinous-offence clusters. This is deliberately a simple, explainable binning approach rather than an opaque clustering model — see Section 10.1 for the rationale.

9.5 Repeat-offender logic

Rows in Accused are grouped by the same normalized-identity key described in 9.3, then filtered to groups spanning more than N distinct CaseMasterID values (configurable; default N=3 to mirror the brief's own example query). Each group's district span (via CaseMaster → Unit → District) is computed to flag cross-district repeat offenders specifically, since those are the pattern most invisible to a single station today.

10. Analytics Engine

All six algorithms below are chosen for the same reason: they are simple enough to implement correctly in the hackathon window, and simple enough to explain to a non-technical judge or officer in one sentence — which matters as much as sophistication for a system whose output may inform investigative or resourcing decisions.

Algorithm	Method	Why this method
Hotspot detection	Geohash / fixed-grid binning of CaseMaster.latitude/longitude, filtered by CrimeSubHead and date range, counted per cell.	Fully explainable (“N cases in this 500m cell in the last 90 days”); no black-box clustering parameters to defend to judges.
Repeat-offender detection	Trigram/Levenshtein similarity on AccusedName + AgeYear tolerance (± 2 yrs) + matching GenderID, clustered into identity groups; count distinct CaseMasterID per group.	Every match carries a similarity score shown to the user; threshold is tunable and disclosed.
Co-accused network	Graph edge = shared CaseMasterID between two Accused rows; centrality via simple degree count (who has the most distinct co-accused / cases).	Degree centrality is easy to compute and explain in a live demo; betweenness/eigenvector centrality is listed as a stretch enhancement only if time allows.
Trend analysis	Weekly/monthly COUNT(CaseMasterID) grouped by CrimeSubHead and District, plotted as a line chart with a trailing moving average.	Moving average is a transparent, well-understood baseline for a 48-hour build.
Anomaly detection	Z-score of the current week's count against the trailing 8-week mean/std-dev per station + crime sub-head; flag $ z > 2$.	Single, well-known formula; the flagged reason is printed in plain language, not just a badge.
Risk / repeat-offender scoring	Composite of: distinct-case count, distinct-district count, network degree centrality. Caste, religion, and occupation are explicitly excluded as model inputs.	Exclusion of protected attributes is a stated design rule, tested with a unit test that fails the build if those columns are referenced in the scoring query.

11. System Architecture

11.1 Components

- Frontend — Next.js (React) + TypeScript + Tailwind + shadcn/ui; Leaflet/Mapbox for hotspot maps; React Flow / Cytoscape.js for the network graph; deployed on Netlify.
- API Gateway — FastAPI (Python), handles auth, RBAC scoping, rate limiting, and routes requests to the AI orchestrator; deployed on Vercel.
- AI Orchestrator — a LangGraph agent with explicit nodes: intent router → schema retriever → SQL generator → SQL validator → executor → explanation packager.
- Query Engine — SQLAlchemy against the active database (Neon Postgres in development, Zoho Catalyst DataStore at final submission), executing only through the SELECT-only role described in Section 8.4.
- Analytics Engine — a Python service (pandas, networkx, geohash) implementing the six algorithms in Section 10, callable both from the AI orchestrator and directly from the map/network/dashboard views.
- Audit Service — a dedicated append-only log store, written to on every query regardless of success or failure, isolated from the case-data tables so it cannot be altered through the same access path as investigative data.

11.2 Data flow, step by step

1. User submits a question (typed or transcribed voice) from the chat UI.
2. API Gateway authenticates the session and resolves the user's role and jurisdiction scope.
3. AI Orchestrator retrieves the relevant schema chunks by embedding similarity (Section 8.1).
4. The LLM generates a candidate SQL statement constrained to the retrieved schema.
5. The SQL validator checks statement type, table/column allow-list, and injects the jurisdiction/row-cap constraints.
6. The query engine executes the validated SQL against the read-only role.
7. Results are shaped for the appropriate visualization (map / graph / table / single value).
8. The explanation packager attaches SQL, tables used, and a confidence score to the response.
9. The audit service logs the full event (user, role, question, SQL, tables, confidence, timestamp).
10. The response streams back to the frontend over WebSocket for a responsive, incremental feel.

12. API Design

POST /api/v1/chat/query

Submits a natural-language question within a session.

```
Request:
{
  "sessionId": "a1b2c3",
  "message": "Show robbery hotspots in Bengaluru East in the last 90 days",
  "lang": "en"
}

Response:
{
  "answer": "There are 42 robbery cases in Bengaluru East in the last 90 days, clustered around 3 hotspots.",
  "sql": "SELECT cm.CaseMasterID, cm.latitude, ... LIMIT 2000;",
  "tablesUsed": ["CaseMaster", "CrimeSubHead", "Unit", "District"],
  "confidence": 0.91,
  "visualizationType": "map",
  "data": { "points": [ { "lat": 12.97, "lng": 77.64, "caseMasterId": 10234 } ] }
}
```

GET /api/v1/network/{accusedId}

Returns the co-accused / repeat-appearance graph rooted at a given accused identity cluster, with a depth parameter.

GET /api/v1/hotspots

Query parameters: district, crimeSubHead, from, to. Returns binned hotspot cells for the map layer.

GET /api/v1/audit/logs

SCRB / SP roles only. Query parameters: from, to, userId (optional). Returns paginated audit entries.

POST /api/v1/export/pdf

Body: { sessionId }. Returns a downloadable PDF of the conversation.

GET /api/v1/auth/me

Returns the authenticated user's role and jurisdiction scope, used by the frontend to gate UI elements.

13. Security & Compliance

13.1 Role-based access control

Roles — SHO, IO, Analyst, SCRB, SP — map to a jurisdiction scope derived from Employee.UnitID / Employee.DistrictID. Query results are automatically filtered to the caller's scope at the SQL-generation layer (an extra WHERE clause is injected, not left to the LLM to remember), except for SCRB and SP roles, which may query state-wide.

13.2 Query restrictions

- SELECT-only, allow-listed tables/columns (Section 8.3).
- ComplainantDetails.CastelID and ComplainantDetails.ReligionID are blocked from any predictive or individual-level query path; they are reachable only through a separate, SCRB-gated aggregate reporting endpoint that returns counts, never row-level records.

13.3 PII masking

Names in ComplainantDetails, Victim, and Accused are shown in full only to roles whose jurisdiction covers the case. In cross-jurisdiction analytical views (e.g. a state-wide trend chart), names are masked (e.g. “A— K—”) unless the viewing role is SCRB/SP, and every access to an unmasked name is itself an audit-logged event.

13.4 Audit logging

Every query — successful, rejected, or errored — is written to the append-only audit store (Section 11.1) with user, role, question text, generated SQL, tables touched, confidence, and timestamp.

13.5 Encryption

TLS in transit for all API and WebSocket traffic; at-rest encryption on the database (Neon and Zoho Catalyst both support this); PII columns additionally encrypted at the application layer where the underlying platform allows.

13.6 Rate limiting

Per-user token-bucket limiting at the API gateway to prevent both abuse and runaway LLM cost.

13.7 Prompt-injection protection

- The SQL-generation prompt only ever contains schema metadata and the user's own question — free-text case data (e.g. CaseMaster.BriefFacts) is never fed back into a prompt that also generates executable SQL, preventing stored-data prompt injection.
- The validator (Section 8.3) is the actual security boundary, not the prompt — the system assumes the LLM output is untrusted until parsed and allow-list-checked.

13.8 A deployment-stack caveat worth stating explicitly

Critical evaluation — hosting choice vs. real police data

Neon, Vercel, Netlify, and Zoho Catalyst are practical choices for a hackathon build and are used here exactly as the brief mandates. They are not, as-is, an endorsement for hosting real KSP FIR data in production: that would need review against India's DPDP Act 2023 obligations and any KSP/CCTNS data-hosting policy, most likely requiring an empanelled government

cloud or NIC-hosted environment. This PRD treats the current stack as a demo/dev environment only, and lists the production hosting question explicitly under Section 6.3 (Future Vision) and Section 16 (Deployment Strategy) rather than glossing over it.

14. Explainable AI Strategy

Every chat response returned by the API (Section 12) carries five explainability fields, always populated, never optional:

- `sql` — the exact, validated SQL statement executed.
- `tablesUsed` — the list of tables/columns the query touched.
- `confidence` — a score combining schema-retrieval match strength, SQL-validation pass/fail, and result-set sanity (e.g. an empty result lowers confidence rather than being presented as a confident “zero”).
- `reasoningSummary` — a one-to-two sentence plain-language restatement of the query plan (“I filtered CaseMaster to Robbery cases in Bengaluru East from the last 90 days and grouped by location”).
- `queriedBy / timestamp` — who asked, and when, mirroring the audit log entry.

In the UI, this is surfaced as a collapsible “Show reasoning” panel attached to every chat answer (Section 15.1), so the default view stays clean for a fast-moving investigation while the full provenance is always one click away for review or dispute.

15. UI/UX Specification

15.1 Investigator Chat

Three-pane layout: a left rail listing past sessions, a center conversation thread (question / answer / inline chart-or-map thumbnail), and a right-hand collapsible “Reasoning” panel showing SQL, tables used, and confidence for the currently-selected answer (Section 14). A persistent input bar at the bottom supports typed text, a language toggle (EN/KN), and a microphone button.

15.2 Crime Map

Full-width Leaflet map with a heatmap layer, a left filter panel (district, station, crime sub-head, date range), and a summary strip above the map (“42 cases, 3 hotspot clusters”). Clicking a cluster opens a side panel listing the underlying CaseMasterID records.

15.3 Network Explorer

Cytoscape.js force-directed graph. Clicking a node expands its direct co-accused/repeat-appearance edges; hovering an edge shows the linking CaseMasterID(s) and, for repeat-appearance edges, the identity-match confidence score (Section 9.3) in a tooltip — never hidden.

15.4 District Dashboard

KPI cards across the top (open FIRs, chargesheet rate, pending-beyond-60-days count), a trend line beneath (Section 10, trend analysis), and a bar chart of top crime sub-heads for the selected district.

15.5 Alert Center

A simple list of anomaly and emerging-hotspot alerts (Section 10), each tagged with a severity indicator and the plain-language reason the alert fired, sorted by recency.

15.6 PDF Export

A single “Export conversation” button in the chat header, generating the PDF described in Section 7.7.

16. Deployment Strategy

16.1 Development environment

- Neon Postgres for development, using a per-feature database branch so the team can iterate on schema/queries without colliding.
- FastAPI backend deployed as Vercel serverless functions.
- Next.js frontend deployed on Netlify.

16.2 Migration path to Zoho Catalyst DB (mandatory final target)

All database access goes through a single SQLAlchemy repository layer (Section 11.1), so the migration surface is one module, not scattered queries. Before migration, every query used by the demo is re-tested against Zoho Catalyst's query layer (ZCQL).

Migration risk to plan for, not discover on demo day

Zoho Catalyst's ZCQL does not have full ANSI-SQL parity with Postgres — window functions and some multi-table join patterns may behave differently or be unsupported. The team should not assume a query that works on Neon will work unchanged on Catalyst. Recommended mitigation: keep the repository layer's SQL simple and portable, run the full use-case test set (Section 5) against both databases before the final submission, and keep the Neon-backed build as a fallback demo environment if a late Catalyst-specific issue appears.

16.3 Compatibility considerations

- Avoid Postgres-only geospatial functions (e.g. earthdistance/PostGIS) in any code path that must also run on Catalyst; compute hotspot distances in the Python analytics layer (haversine formula) instead of in-database GIS functions, so the same code runs against either backend.
- Keep generated SQL from the AI orchestrator to standard SELECT/JOIN/GROUP BY/HAVING constructs matching the allow-list in Section 8.3, which are the constructs most likely to be portable.

17. Hackathon MVP Plan (48 Hours)

Every deliverable below is scoped to be independently demoable, so that if time runs short after Phase 3, the team still has a coherent, working product rather than a pile of half-finished features.

Phase	Focus	Deliverable
Phase 1 (6 hrs)	Schema ingestion + embeddings for retrieval; auth/RBAC skeleton; Neon DB seeded with synthetic sample data (no real FIR data used in the demo).	Working auth + seeded dev database + queryable schema embeddings.
Phase 2 (12 hrs)	NL-to-SQL pipeline (LangGraph agent) + validator wired end-to-end for English queries against the core tables; basic chat UI.	Chat answers the first 10 use cases (Section 5) correctly with SQL shown.
Phase 3 (12 hrs)	Hotspot map, co-accused network graph, and repeat-offender detection algorithm implemented and wired to the chat/UI.	Map and network pages functional against the seeded dataset.
Phase 4 (12 hrs)	Explainability panel, audit log, PDF export, trend/anomaly analytics; Kannada input (best-effort) and voice input (Web Speech API) if on schedule.	Full MVP feature set (Section 6.1) wired and demoable end-to-end.
Final polish (6 hrs)	Bug bash; richer demo data; rehearse the demo script (Section 18); deploy to Netlify/Vercel; one-page architecture slide.	Stable, rehearsed demo build deployed and ready.

18. Demo Script (5 Minutes)

Segment	What happens
Opening (0:00–0:30)	Frame the problem: 1,100+ stations, static dashboards, and a question an officer actually needs answered today that no dashboard covers.
Live query (0:30–1:30)	Type “Show robbery hotspots in Bengaluru East in the last 90 days” live — SQL and map appear within seconds.
Hotspot map (1:30–2:15)	Zoom into a cluster, change the date range live, show the filter combination working.
Criminal network (2:15–3:15)	Click an accused node from a burglary case, expand co-accused, surface a cross-district repeat-offender link with its confidence score visible.
Predictive alert (3:15–4:00)	Open the Alert Center, show an emerging-hotspot or anomaly card with its plain-language reason.
Explainability panel (4:00–4:45)	Open the “Show reasoning” panel on the first answer — SQL, tables used, confidence — and tie it explicitly to why this is usable in a police context, not just a chatbot demo.
Closing impact (4:45–5:00)	One line: officer time saved on routine queries, and a cross-district pattern surfaced today that would otherwise stay invisible.

19. Judging Alignment Matrix

Feature	Innovation	Impact	Feasibility	Scalability	Security
NL-to-SQL conversational chat	High	High	High	High	Medium – mitigated by validator
Criminal network graph	High	High	Medium	Medium	Medium – confidence-scored
Hotspot map	Medium	High	High	High	Low risk
Explainability panel / audit trail	High	High	High	High	High – core to trust
Kannada support	High	Medium	Medium (stretch)	Medium	Low risk
RBAC + jurisdiction scoping	Medium	High	High	High	High
Trend & anomaly detection	Medium	Medium	High	High	Low risk

20. Final Recommendation

20.1 Strongest differentiator

The explainability and audit layer is the feature most worth protecting under time pressure. It is what turns “a chatbot over crime data” into something a police oversight process can actually trust: every answer is traceable to the exact SQL and tables that produced it, every query is logged, and identity-matching uncertainty (Section 9.3) is surfaced rather than hidden. Judges evaluating for real-world usefulness and security/privacy will notice this before anything else.

20.2 What to cut first if time runs short

- Voice input — nice for the demo, not central to the product claim.
- Full Kannada generation — keep the translation-layer fallback (Section 7.2) rather than attempting native bilingual grounding under time pressure.
- Forecast sophistication — the simple moving-average and z-score methods (Section 10) are sufficient; do not spend hackathon hours on a more elaborate model.

20.3 What judges are most likely to remember

An officer typing a plain-English (or Kannada) question and watching SQL, a hotspot map, and a criminal-network graph populate live — followed immediately by a transparent “here is exactly how I know this, and here is how confident I am” panel. That sequence, in under two minutes, demonstrates innovation, real-world usefulness, and the security/trust posture a police data product specifically requires, in a single continuous demo beat.